

Experiment No.10

Aim: To study and Implement Containerization using Docker on AWS/Azure/Google cloud platform.

Objectives:

- Develop a theoretical understanding of containerization principles, focusing on Docker as a containerization tool and its benefits for application deployment and management.
- Apply practical knowledge to configure and utilize Docker on AWS, emphasizing container creation, deployment, and orchestration.

Outcomes:

- To demonstrate proficiency in utilizing Docker for containerization on the AWS cloud platform.
- Apply knowledge in deploying and orchestrating containers on AWS, understanding the advantages of containerization for scalable and portable application deployment.

Software Requirements: Windows/Ubuntu OS, AWS Account, Docker.

Theory:

Containerization is a lightweight and portable solution for packaging, distributing, and running applications. Docker is a leading containerization platform that allows developers to package applications and their dependencies into containers. Containers provide consistency across different environments and simplify the deployment process.

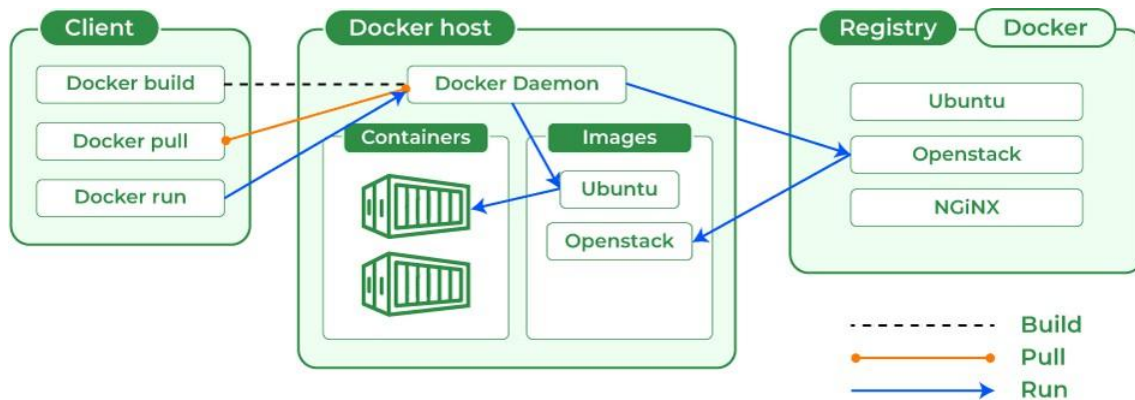


Figure 10 Docker architecture

Key Components:

1. **Docker Engine:** The heart of Docker, responsible for creating, running, and managing containers.
2. **Docker Images:** Read-only templates containing application code, dependencies, and configuration needed to run a container.
3. **Docker Containers:** Isolated, lightweight environments that package an application with its own operating system, libraries, and code.
4. **Docker Registry:** A repository for storing and sharing Docker images (e.g., Docker Hub).
5. **Docker Client:** Command-line interface (CLI) or graphical user interface (GUI) used to interact with Docker.

How Docker Works:

1. **Image Building:** Create a Docker image using a Docker file, which specifies instructions for building the image.
2. **Image Sharing:** Push images to a Docker registry for sharing and collaboration.
3. **Container Creation:** Run a container from an image using the **docker run** command.
4. **Container Isolation:** Each container runs in its own isolated environment, sharing the host kernel but having its own filesystem, processes, and network.
5. **Resource Management:** Docker efficiently manages resources, allocating CPU, memory, and storage to containers.
6. **Portability:** Docker containers can run on any system with Docker installed, ensuring consistency across environments.

Benefits of Docker:

- **Portability:** Move containers seamlessly between different environments (development, testing, production).
- **Isolation:** Containers run independently, preventing conflicts and enhancing security.
- **Resource Efficiency:** Containers share the host kernel, reducing resource overhead compared to virtual machines.
- **Agility:** Spin up and tear down containers quickly, enabling rapid deployment and scaling.
- **Consistency:** Ensure consistent application behavior across environments.
- **Simplified Development:** Streamline development and testing processes with Docker.

Amazon Elastic Container Service (ECS) is an AWS service for running Docker containers on a scalable and managed infrastructure. It simplifies the deployment and management of containers, providing features such as automatic scaling, load balancing, and integration with other AWS services.

Stepwise Procedure to Implement PaaS using AWS Elastic Beanstalk:

Step 1: Sign into AWS Console

- Log in to the AWS Management Console using your AWS account credentials.

Step 2: Install Docker on Local Machine

- Install Docker on the local development machine.

Step 3: Create Docker Image

- Create a Docker file to define the application and its dependencies.
- Build a Docker image using the Docker file.

Step 4: Push Docker Image to Amazon ECR

- Create an Amazon Elastic Container Registry (ECR) repository.
- Push the Docker image to the ECR repository.

Step 5: Create ECS Cluster

- Navigate to the ECS service in the AWS Console.
- Create an ECS cluster to host the Docker containers.

Step 6: Define ECS Task Definition

- Define an ECS Task Definition, specifying container details and resource requirements.

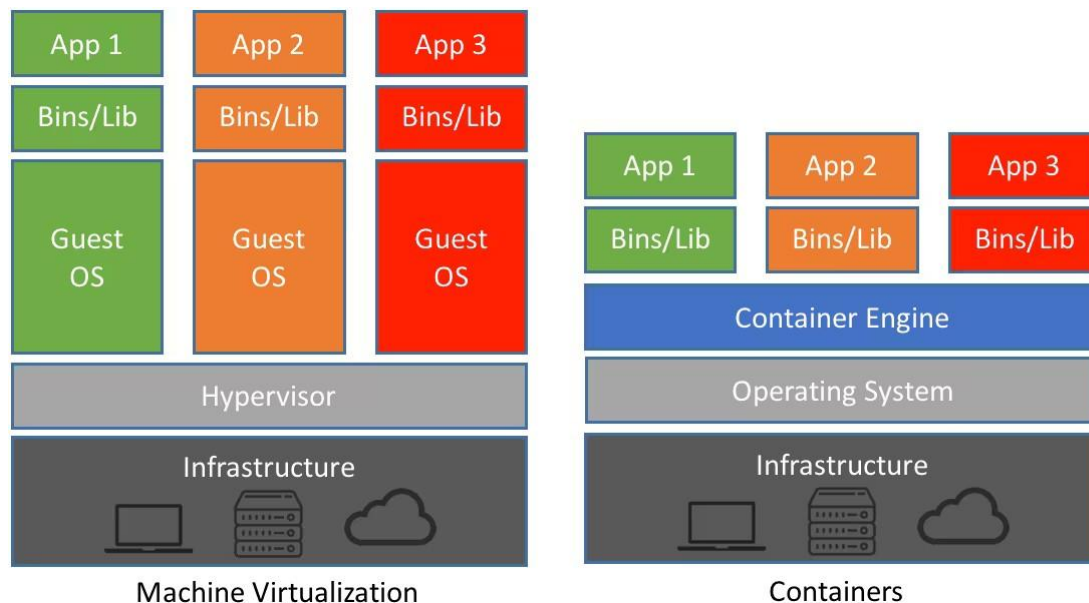
Step 7: Run Docker Container on ECS

- Run the Docker container on the ECS cluster.
- Monitor the container's status and logs in the ECS Console.

Step 8: Scale and Update Containers

- Scale the number of running containers to demonstrate automatic scaling.
- Update the Docker image and redeploy the containers.

Comparison between Virtualization Vs Container



Implementation

- Create an account on docker hub (<https://hub.docker.com/>)
- Open <https://labs.play-with-docker.com/>
- Login with docker hub credentials, create new instance.
- Container is a runnable (runtime) instance of an image.

Commands:

\$ docker version

Shows community edition which is free - server (actual docker) and client (CLI) version and another is Enterprise version

```
$ docker
$ docker container--help
$ docker container ls
$ docker container run --publish 8080:80 -d nginx
$ docker container ls
$ docker container run --publish 8081:80 -d--name nginx-demo nginx
$ docker container ls
```

Start/Stop Container:

```
$ docker container stop (initial 4 values)
$ docker container ls -a (to list all the container)
$ docker container start "name of the container"
```

To remove the container:

```
$ docker container rm nginx-demo (can't remove container if it is running)
```

To remove it forcefully:

```
$ docker container rm -f nginx-demo
$ docker container ls
$ docker container ls -a
```

To see the statistics

```
$ docker container stats
```

To see the images on your docker

```
$ docker image ls
$ docker image history nginx
```

Let's try to create own image

```
FROM nginx:latest
WORKDIR /usr/share/nginx/html
COPY index.html
```

```
$ curl www.google.com > index.html
```

```
$ ls
Shows index.html
```

```
$ touch DockerFile
$ chmod 777 DockerFile
$ ls
Shows docker file and index.html
```

Use editor option

Copy/paste content in DockerFile and Save option.

Using this we are using custom based image so will have 2 images 1 is default image and one custom based which will show our webpage instead of default one

On command prompt

```
$ docker image build -t custom-nginx-html -f DockerFile. (dot to copy file in root)
```

```
$ docker image ls (shows 2 configuration)
```

```
$ docker container run --publish 8080:80 --name original-nginx -d nginx
```

```
$ docker container run --publish 8081:80 --name my-nginx -d custom-nginx-html
```

```
$ docker container ls
```

Can check clicking on 8080 shows default nginx page and 8081 shows google page

To handover custom image

```
$ docker image ls
```

Handover custom image file, another person runs docker and he will be able to run the container.

Conclusion:

Thus, we have understood and implemented containerization using Docker on the AWS cloud platform.